CHANDIGARH UNIVERSITY
Discover. Learn. Empower.

Experiment – 9

Name: Prionti Das
UID: 24BAI71005
Branch:CSE(AIML)
Section/Group: 24AIT_NTPP-3[B]
Subject: Full Stack - II
Date of Performance:
Subject Code: 24CSP-337
Submitted to: Ms.Lovleen Kaur(E20454)

EXPERIMENT – 9.1

1. **AIM:** To design and implement a Continuous Integration (CI) pipeline that automates application building and testing using unit testing, integration testing, contract testing, and automated workflows to improve software quality and reliability.
2. **OBJECTIVE:**

    1. To understand the concepts and importance of Continuous Integration (CI) in software development.
    2. To implement unit and integration testing to verify application functionality and ensure that different components work correctly together.
    3. To perform contract testing using Pact to verify communication and compatibility between different services.
    4. To automate testing workflows so that applications are built and tested automatically whenever code changes are pushed to the repository.

3. **SOFTWARE REQUIREMENTS:**
    1. Spring Boot
    2. Apache Maven
    3. Docker
    4. MockMvc
    5. Visual Studio Code
4. **THEORY:**

Continuous Integration (CI) is a software development practice in which code changes are automatically built and tested whenever developers push updates to a shared repository. It helps detect errors early and improves software quality.

Unit testing uses JUnit 5 to verify individual methods, while integration testing with MockMvc checks whether application components and REST APIs work correctly. Contract testing using Pact verifies that different services communicate according to agreed requirements. Test containers provide temporary environments for testing applications with real dependencies, such as databases or Kafka.

By using GitHub Actions to automate the build and testing process, every code change can be validated automatically. This reduces manual effort, detects defects early, and improves application reliability.

5. **IMPLEMENTATION/PROCEDURE:**
    1. Install and configure Java JDK, Spring Boot, Maven, Git, and Docker.
    2. Create a Spring Boot project and add the required testing dependencies.
    3. Write unit tests using JUnit 5 to verify individual methods.
    4. Implement API integration tests using MockMvc to test REST endpoints.

24BAI71005_Prionti

5. Configure contract tests using Pact to verify communication between services.
6. Use Testcontainers to test application components with real dependencies, such as databases or Kafka.
7. Create a GitHub repository and configure a GitHub Actions workflow to build the application and run tests automatically whenever code is pushed.
8. Push the code to GitHub and verify the build status and test results in the Actions tab.
9. Fix any failed tests and rerun the pipeline to ensure the application builds and tests successfully.

6. CODE :

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>NexaStock | Overview</title>
  <link rel="stylesheet" href="style.css">
</head>
<body>
<div class="app-layout">
  <aside class="sidebar">
    <div class="brand">
      <div class="brand-logo">N</div>
      <div>
        <h2>NexaStock</h2>
        <small>INVENTORY WORKSPACE</small>
      </div></div>
    <p class="sidebar-label">WORKSPACE</p>
    <nav>
      <a class="nav-link active" href="index.html">▦   Overview</a>
      <a class="nav-link" href="inventory.html">▤   Inventory</a>
      <a class="nav-link" href="profile.html">👤   Student Profile</a>
    </nav>
  </aside>
  <main class="main-content">
    <div class="breadcrumb">Workspace / Overview</div>

    <header class="page-header">
      <div>
        <h1>Overview</h1>
        <p>Welcome back! Here's your workspace summary.</p>
      </div>
      <a href="inventory.html" class="primary-button">View Inventory →</a>
    </header>
    <section class="stats-grid">
      <div class="stat-card">
        <p>Total Inventory Items</p>
        <h2 id="totalItems">—</h2>
      </div>
      <div class="stat-card">
        <p>Workspace</p>
        <h2>NexaStock</h2>
      </div>
      <div class="stat-card">
```

24BAI71005_Prionti

```
            <p>Student</p>
            <h2>Prionti Das</h2>
          </div></section>
        <section class="content-card">
          <h2>Welcome to NexaStock</h2>
          <p class="muted">
            Manage inventory items, explore your workspace, and access
            your student information using the navigation menu.
          </p>
          <a href="inventory.html" class="primary-button">Manage Inventory</a>
        </section>
      </main></div>
<script>
fetch("/api/items")
  .then(response => {
    if (!response.ok) throw new Error("API unavailable");
    return response.json();
  })
  .then(items => {
    document.getElementById("totalItems").textContent = items.length;
  })
  .catch(() => {
    document.getElementById("totalItems").textContent = "—";
  });
</script>
</body>
</html>
```

24BAI71005_Prionti

7. OUTPUT:

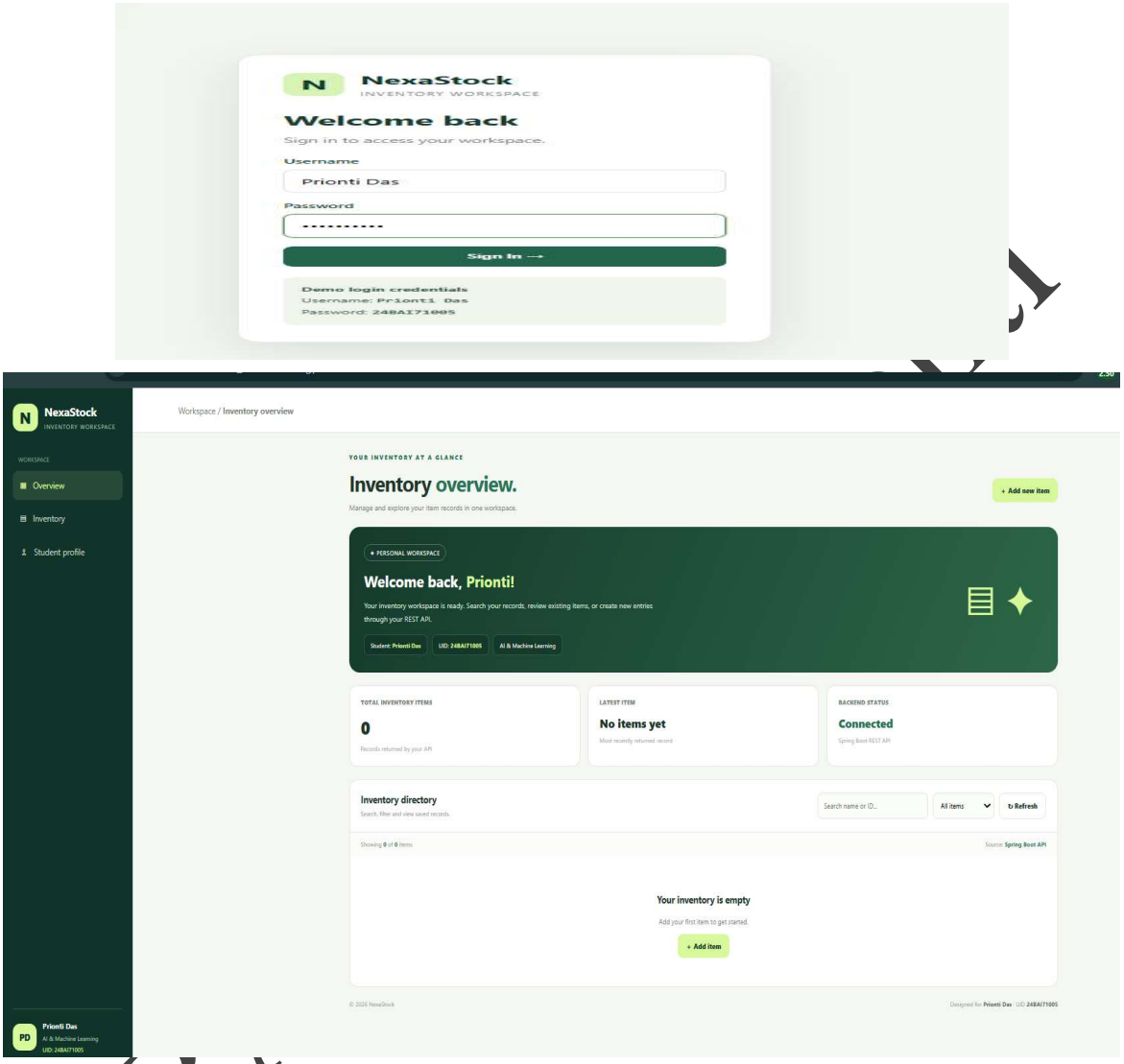


Fig 9.1:NexaStock Inventory Management System Interface

The NexaStock Inventory Management System provides a user-friendly login page and an interactive dashboard for managing inventory records. Users can sign in using their credentials and view the inventory overview, total number of items, latest item, and backend connection status. The dashboard also provides options to add new items, search records, filter inventory, and refresh the data through the Spring Boot REST API. In the displayed output, the backend is connected successfully, but the inventory is currently empty.

CHANDIGARH UNIVERSITY
Discover. Learn. Empower.

8. LEARNING OUTCOME:

- Understand the concepts and benefits of Continuous Integration (CI).
- Implement unit and integration testing using JUnit 5 and MockMvc.
- Perform contract testing using Pact to verify service compatibility.
- Use Testcontainers to test applications with real dependencies.
- Configure an automated CI pipeline using GitHub Actions to build and test applications.

24BAI71005_Prionti

CHANDIGARH UNIVERSITY
Discover. Learn. Empower.

EXPERIMENT – 9.2

1. **AIM:** To design and implement a Continuous Deployment (CD) pipeline using Docker containerization, GitHub Actions, and automated release strategies, including canary deployment, to ensure faster, safer, and more reliable application delivery.
2. **OBJECTIVE:**

    1. To understand the concepts and importance of Continuous Deployment (CD) in software development.
    2. To containerize Spring Boot applications using Docker to ensure consistent execution across different environments.
    3. To automate application deployment using GitHub Actions by configuring workflows for building, testing, and releasing applications.
    4. To implement a canary deployment strategy by releasing new application versions to a small group of users before expanding deployment to all users.

3. **SOFTWARE REQUIREMENTS:**
    1. Docker
    2. Spring Boot
    3. Git and GitHub
    4. Apache Maven
    5. Visual Studio Code
4. **THEORY:**

    Continuous Deployment (CD) is a software development practice in which application updates are automatically deployed after successfully passing the required tests. It reduces manual effort and enables faster software delivery.

    Docker containerization packages a Spring Boot application and its dependencies into a container, ensuring consistent execution across different environments. GitHub Actions automates the build, testing, and deployment process whenever code changes are pushed to the repository.

    Canary deployment releases a new application version to a small group of users first. If the new version works correctly, it is gradually released to more users. This reduces deployment risks and helps identify errors before they affect all users.

    By combining Docker, GitHub Actions, and canary deployment, applications can be delivered more efficiently, consistently, and reliably.

5. **IMPLEMENTATION/PROCEDURE:**
    1. Install and configure Java JDK, Spring Boot, Docker, Git, and Maven.
    2. Create a Spring Boot application and verify that it runs successfully.
    3. Create a `Dockerfile` to package the application into a Docker image.
    4. Build the Docker image and test the container locally.
    5. Create a Docker Hub account and push the image to the container registry.
    6. Create a GitHub repository and upload the application source code.
    7. Configure a GitHub Actions workflow to automate building, testing, and publishing the Docker image.
    8. Configure the deployment environment to deploy the application automatically after successful testing.
    9. Implement canary deployment by releasing the new version to a small group of users before expanding it to all users.
    10. Verify the deployment, check application logs, and confirm that the new version works correctly.

24BAI71005_Prionti

6. CODE :

```
<xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
        xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
        xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 https://maven.apache.org/xsd/maven-4.0.0.xsd">
    <modelVersion>4.0.0</modelVersion>
    <groupId>com.experiment</groupId>
    <artifactId>cd-canary-demo</artifactId>
    <version>1.0.0</version>
    <name>CD Canary Deployment Demo</name>
    <description>Continuous deployment with Docker and canary releases</description>
    <parent>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-parent</artifactId>
        <version>3.4.5</version>
        <relativePath/>
    </parent>
    <properties>
        <java.version>17</java.version>
    </properties>
    <dependencies>
        <!-- Spring Boot Web -->
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-web</artifactId>
        </dependency>
        <!-- Spring Boot Actuator -->
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-actuator</artifactId>
        </dependency>
        <!-- Spring Boot Testing -->
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-test</artifactId>
            <scope>test</scope>
        </dependency>
    </dependencies>
    <build>
        <finalName>cd-canary-demo</finalName>
        <plugins>
            <plugin>
                <groupId>org.springframework.boot</groupId>
                <artifactId>spring-boot-maven-plugin</artifactId>
            </plugin>
        </plugins>
    </build>
</project>
```

24BAI71005_Prionti

7. OUTPUT:

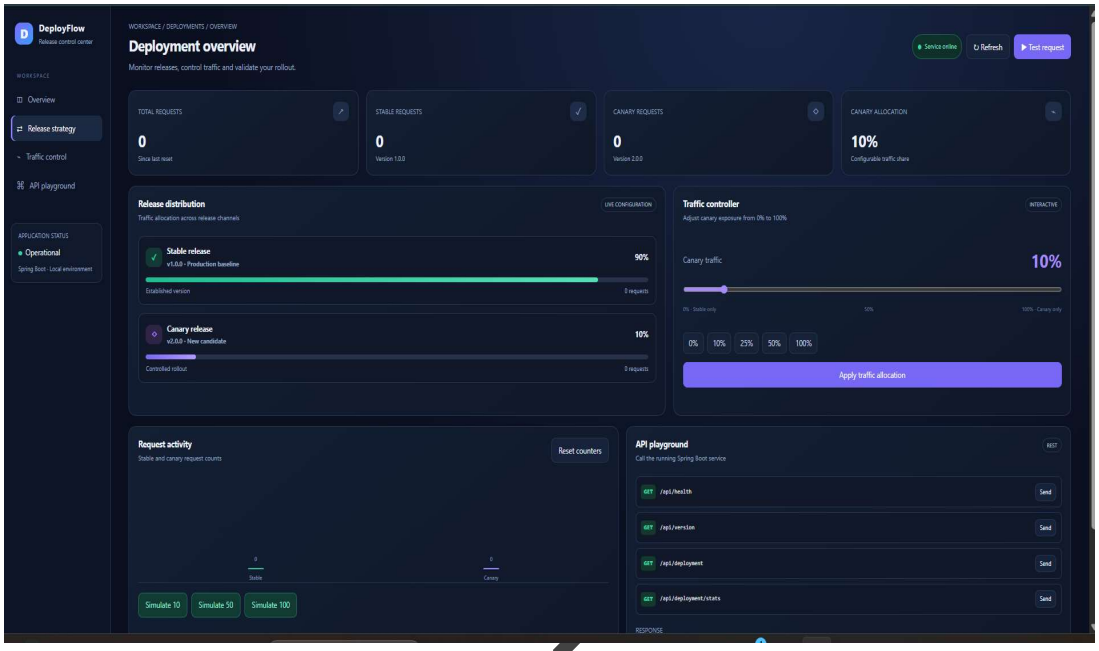


Fig 9.2: DeployFlow Continuous Deployment Dashboard with Canary Release Strategy

The DeployFlow dashboard provides an interface for monitoring application releases and managing canary deployments. It displays the total requests, stable requests, canary requests, and traffic allocation between application versions. In the shown output, 90% of traffic is assigned to the stable release (Version 1.0.0), while 10% is assigned to the canary release (Version 2.0.0). The traffic controller allows users to adjust traffic allocation, and the request activity section helps monitor traffic distribution. The API playground provides options to test the Spring Boot service endpoints. This dashboard demonstrates controlled, automated, and safer application releases using the canary deployment strategy.

8. LEARNING OUTCOME:

- Understand the concepts and benefits of Continuous Deployment (CD).
- Containerize Spring Boot applications using Docker.
- Automate application building and deployment using GitHub Actions.
- Publish Docker images to a container registry.
- Implement canary deployment to release new application versions safely and reliably.

24BAI71005